CS266 - Database Management Systems

University Accommodation Office Management System

Presentation and Technical Report (LaTeX)

Project Type:	Full-stack DBMS Application
Frontend:	React + TypeScript + Vite
Backend:	FastAPI + SQLAlchemy Core
Database:	MySQL 8+

Team Roll Numbers

24bcs001
24bcs002
24bcs003
24bcs004
24bcs005
24bcs006
24bcs007

Date: April 7, 2026

Contents

1 Executive Summary **2**

2 Problem Statement and Objectives **2**
2.1 Problem Context . . . 2
2.2 Project Objectives . . . 2

3 Complete Tech Stack **2**
3.1 Frontend . . . 2
3.2 Backend . . . 3
3.3 Database and Testing . . . 3

4 Repository Structure and Responsibilities **3**

5 System Architecture from Start to End **4**
5.1 High-Level Flow . . . 4
5.2 Development Proxy and Service Boundaries . . . 4

6 Database Design (Relational Core) **4**
6.1 Primary Entities . . . 4
6.2 ER Diagram Placeholder . . . 5
6.3 Important Relational Rules . . . 5
6.4 Data Integrity Controls . . . 5
6.5 Report Views (DBMS-Oriented) . . . 5

7 How Backend is Connected to DB **6**
7.1 Environment Loading . . . 6
7.2 DB Settings Used . . . 6
7.3 SQLAlchemy + PyMySQL Connection URL . . . 6
7.4 Engine and Session Lifecycle . . . 6
7.5 Metadata Reflection Strategy . . . 6

8 Authentication and Authorization Internals **7**
8.1 Auth Model . . . 7
8.2 Seeded Demo Accounts . . . 7
8.3 Role Guards . . . 7

9 Frontend Process Flow from Scratch **7**
9.1 Router and Guards . . . 7
9.2 API Wrapper Behavior . . . 7
9.3 Session Persistence . . . 7

10 Complete API Inventory and Operations **7**
10.1 Core Endpoints . . . 7
10.2 Report Endpoints (a to n) . . . 8

11 API Calling Examples (Complete Operational Coverage) **8**
11.1 Step 1: Login and Capture Token . . . 8

11.2 Step 2: Read Current User Profile . . . 9
11.3 Step 3: CRUD Operations Example (students) . . . 9
11.4 Step 4: Report API Examples . . . 10

12 Detailed Internal Process Flows **10**
12.1 Flow A: Login Process . . . 10
12.2 Flow B: Generic Create Operation . . . 10
12.3 Flow B1: Generic List Route for Each Entity (`GET /api/{entity}`) . . . 10
12.4 Flow C: Report Query Execution . . . 12

13 How Type Normalization and Validation Work **12**

14 Error Handling Strategy **12**
14.1 Constraint and Integrity Mapping . . . 12
14.2 Transaction Safety . . . 12

15 ACID Properties: Theory and Implementation in This Project **13**
15.1 Important Clarification . . . 13
15.2 A - Atomicity . . . 13
15.3 C - Consistency . . . 13
15.4 I - Isolation . . . 13
15.5 D - Durability . . . 13
15.6 Practical Note on SQLAlchemy . . . 13

16 Code-Level Examples from Real Project **13**
16.1 FastAPI Route Example . . . 13
16.2 Generic CRUD Example . . . 14
16.3 Frontend API Wrapper Example . . . 14

17 Report Layer and DBMS Demonstration Value **14**

18 Deep Execution Pipeline: From `app.get(...)` to `session.execute(...)` **15**
18.1 ASGI-to-Route Lifecycle (Exact Runtime Sequence) . . . 15
18.2 How `app.get(...)` is Called Internally . . . 15
18.3 CRUD Query Object Construction (SQLAlchemy Core) . . . 15
18.4 Report Query Construction (Textual SQL + Bind Params) . . . 16
18.5 Write Operations and Transactions . . . 16

19 All Views Explained in Detail (Complete Report Layer) **16**

20 Complete Per-Table Technical Specification (All 10 Tables) **17**
20.1 `staff` . . . 17
20.2 `courses` . . . 17
20.3 `students` . . . 17
20.4 `halls` . . . 18
20.5 `apartments` . . . 18
20.6 `rooms` . . . 18
20.7 `leases` . . . 19
20.8 `invoices` . . . 19

20.9 `next_of_kin` 19
20.10 `inspections` 19

21 Foreign Key Matrix (All Tables) 20

22 Index Catalog and Query Optimization Impact 20

23 Deployment and Runtime Notes 21
23.1 Local Execution 21
23.2 Cloud Split Deployment 22
23.3 TLS and Managed DB Logs 22

24 Testing and Quality Assurance 22
24.1 Route-Level Tests 22
24.2 Runtime Smoke Test 22
24.3 What Is Validated 22

25 Known Constraints and Future Improvements 22

26 Final Conclusion 22

A Full Report Endpoint Quick List 23

B Generic CRUD Pattern Quick List 23

1 Executive Summary

This project is a complete University Accommodation Office Management System designed as a DBMS-centric full-stack application. It solves a practical institutional problem: tracking students, staff, housing inventory, leases, invoices, compliance inspections, and emergency contacts in one consistent relational system.

The implementation is split into three layers:

- **Database layer (MySQL):** normalized schema, constraints, triggers, indexes, and report views.
- **Service layer (FastAPI + SQLAlchemy):** secure APIs for authentication, CRUD, and reporting.
- **Presentation layer (React + TypeScript):** login, entity studio, dashboards, and report execution UI.

This report explains the project from scratch to deployment, including every process flow, all API operations, DB connectivity, transaction handling, and how ACID properties are realized in practice.

2 Problem Statement and Objectives

2.1 Problem Context

Accommodation offices often suffer from fragmented data and manual workflows:

- Student allocation data in spreadsheets.
- Lease and invoice records in disconnected systems.
- Slow or inconsistent reporting.
- Weak integrity checks that allow invalid data.

2.2 Project Objectives

1. Build a normalized relational schema for accommodation operations.
2. Provide secure, role-based APIs for operational workflows.
3. Support complete CRUD across all key entities.
4. Implement DBMS-style report queries via SQL views.
5. Provide a clean, modern frontend for daily usage.
6. Ensure data consistency through constraints, triggers, and transaction handling.

3 Complete Tech Stack

3.1 Frontend

- React 18.3.1
- TypeScript 5.7.2

- Vite 6.2.1
- React Router DOM 6.30.0
- Tailwind CSS 3.4.17
- Framer Motion 12.9.4
- Lucide React 0.469.0

3.2 Backend

- FastAPI 0.115.12
- Uvicorn 0.34.0
- SQLAlchemy 2.0.40 (Core style + reflection)
- PyMySQL 1.1.1
- python-dotenv 1.0.1
- python-jose 3.4.0
- passlib 1.7.4

3.3 Database and Testing

- MySQL 8+ (or compatible managed MySQL)
- pytest 8.3.5
- httpx 0.28.1

4 Repository Structure and Responsibilities

Listing 1: Project layout at high level

```
DBMS_courseproject/
  backend/
    app/
      main.py       # FastAPI routes: auth, reports, generic CRUD
      auth.py       # JWT auth, role checks, seeded users
      config.py     # environment loading and settings parsing
      database.py   # engine, session, metadata reflection
    schema.sql      # tables, FKs, triggers, indexes, views
    requirements.txt
    tests/
  frontend/
    src/
      lib/api.ts          # fetch wrapper + token injection
      providers/AuthProvider.tsx
      config/entities.ts  # UI metadata for all entities
      config/reports.ts   # report catalog and parameters
      pages/
    vite.config.ts
  seed.sql
```

```
  docs/
```

5 System Architecture from Start to End

5.1 High-Level Flow

1. User interacts with frontend page (login, CRUD, reports).
2. Frontend calls API helper in `frontend/src/lib/api.ts`.
3. Request reaches FastAPI route in `backend/app/main.py`.
4. Dependencies validate token and role (if required).
5. Backend executes SQL via SQLAlchemy session.
6. MySQL returns result or constraint error.
7. Backend serializes response into JSON.
8. Frontend renders result in UI.

5.2 Development Proxy and Service Boundaries

Vite proxy routes local requests to backend:

Listing 2: Vite dev proxy

```
proxy: {
  "/api": "http://localhost:8000",
  "/health": "http://localhost:8000",
  "/auth": "http://localhost:8000"
}
```

6 Database Design (Relational Core)

6.1 Primary Entities

The core schema contains 10 transactional tables:

1. staff
2. courses
3. students
4. halls
5. apartments
6. rooms
7. leases
8. invoices
9. next_of_kin
10. inspections

6.2 ER Diagram Placeholder

ER Diagram Placeholder

Add your ER diagram image file at: `docs/figures/erd.png`
Replace this box with:
`\includegraphics[width=0.95\textwidth]{docs/figures/erd.png}`

Figure 1: Entity Relationship Diagram (to be inserted)

6.3 Important Relational Rules

- **students.course_number** references courses with `ON DELETE SET NULL`.
- **students.advisor_staff_id** references staff with `ON DELETE SET NULL`.
- **leases.banner_id** references students with `ON DELETE CASCADE`.
- **invoices.lease_id** references leases with `ON DELETE CASCADE`.
- **next_of_kin.banner_id** is UNIQUE, enforcing max one kin row per student.

6.4 Data Integrity Controls

- Check constraints:
  - apartment bedrooms in {3,4,5}
  - room monthly_rent ≥ 0
  - invoice amount_due ≥ 0
- Trigger-based logic in `rooms`:
  - exactly one of hall_id or apartment_id must be non-null
  - enforced on both INSERT and UPDATE

6.5 Report Views (DBMS-Oriented)

The schema defines 14 views for report routes:

- v_hall_managers, v_student_leases, v_summer_leases
- v_student_rent_paid, v_unpaid_invoices
- v_unsatisfactory_inspections, v_hall_student_rooms

- v_waiting_list, v_student_category_counts
- v_students_without_kin, v_student_advisers
- v_hall_rent_stats, v_hall_place_counts, v_senior_staff

7 How Backend is Connected to DB

7.1 Environment Loading

Listing 3: Environment initialization in config.py

```
BACKEND_ROOT = Path(__file__).resolve().parents[1]
load_dotenv(BACKEND_ROOT / ".env")
load_dotenv(override=False)
```

7.2 DB Settings Used

- `DB_HOST`
- `DB_PORT`
- `DB_USER`
- `DB_PASSWORD`
- `DB_NAME`
- `DB_TLS_MODE` (false, preferred, skip-verify)
- `DB_CA_CERT_PATH`

7.3 SQLAlchemy + PyMySQL Connection URL

Listing 4: Connection URL construction

```
return (
    f"mysql+pymysql://{user}:{password}@{settings.db_host}:{settings.
        db_port}/"
    f"{settings.db_name}?charset=utf8mb4"
)
```

7.4 Engine and Session Lifecycle

- Lazy singleton engine creation in `get_engine()`.
- Connection resilience via `pool_pre_ping=True` and `pool_recycle=300`.
- Per-request session from `sessionmaker`, then session closed in `finally`.

7.5 Metadata Reflection Strategy

Instead of hardcoding ORM models per table, backend reflects metadata at runtime and fetches tables by name. This enables generic CRUD for all entities with one route pattern.

8 Authentication and Authorization Internals

8.1 Auth Model

- OAuth2 bearer token in request header.
- JWT payload contains `sub`, `role`, `exp`.
- Password verification with passlib `pbkdf2_sha256`.

8.2 Seeded Demo Accounts

- admin / Admin@123 (role: admin)
- manager / Manager@123 (role: manager)
- viewer / Viewer@123 (role: viewer)

8.3 Role Guards

- READ roles: admin, manager, viewer
- WRITE roles: admin, manager
- DELETE role: admin only

9 Frontend Process Flow from Scratch

9.1 Router and Guards

- `/login` is wrapped in `LoginOnly`.
- Main app routes are wrapped in `RequireAuth`.
- Unauthorized users are redirected to login.

9.2 API Wrapper Behavior

Listing 5: Auth token injection in api.ts

```
if (!skipAuth && authToken && !headers.has("Authorization")) {
  headers.set("Authorization", `Bearer ${authToken}`);
}
```

9.3 Session Persistence

AuthProvider stores token and user in localStorage and refreshes profile from `/auth/me` on app bootstrap.

10 Complete API Inventory and Operations

10.1 Core Endpoints

Method	Endpoint	Access	Purpose
GET	/health	Public	Service health probe
POST	/auth/login	Public	Issue JWT token
GET	/auth/me	Any authenticated role	Current user profile
GET	/api/{entity}	read roles	List records
GET	/api/{entity}/{id}	read roles	Get single record
POST	/api/{entity}	write roles	Create record
PUT	/api/{entity}/{id}	write roles	Update record
DELETE	/api/{entity}/{id}	admin	Delete record

Supported entities in generic CRUD:

- staff, courses, students, halls, apartments
- rooms, leases, invoices, next-of-kin, inspections

10.2 Report Endpoints (a to n)

ID	Endpoint	SQL View
a	/api/reports/hall-managers	v_hall_managers
b	/api/reports/student-leases	v_student_leases
c	/api/reports/summer-leases	v_summer_leases
d	/api/reports/student-rent-paid/{banner_id}	v_student_rent_paid
e	/api/reports/unpaid-invoices?due_before=...	v_unpaid_invoices
f	/api/reports/unsatisfactory-inspections	v_unsatisfactory_inspections
g	/api/reports/hall-student-rooms/{hall_id}	v_hall_student_rooms
h	/api/reports/waiting-list	v_waiting_list
i	/api/reports/student-category-counts	v_student_category_counts
j	/api/reports/students-without-kin	v_students_without_kin
k	/api/reports/student-adviser/{banner_id}	v_student_advisers
l	/api/reports/rent-stats	v_hall_rent_stats
m	/api/reports/hall-place-counts	v_hall_place_counts
n	/api/reports/senior-staff	v_senior_staff

11 API Calling Examples (Complete Operational Coverage)

11.1 Step 1: Login and Capture Token

Listing 6: Authenticate and obtain bearer token

```
curl -X POST http://localhost:8000/auth/login \
```

```
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"Admin@123"}'
```

11.2 Step 2: Read Current User Profile

```
curl http://localhost:8000/auth/me \
  -H "Authorization: Bearer <TOKEN>"
```

11.3 Step 3: CRUD Operations Example (students)

Listing 7: Create student

```
curl -X POST http://localhost:8000/api/students \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{
    "banner_id":"B00999999",
    "first_name":"Aanya",
    "last_name":"Sharma",
    "street":"1 Main St",
    "city":"London",
    "postcode":"SW1A 1AA",
    "dob":"2003-06-21",
    "gender":"Female",
    "category":"Postgraduate",
    "nationality":"Indian",
    "status":"Waiting",
    "major":"Computer Science"
  }'
```

Listing 8: List students

```
curl http://localhost:8000/api/students \
  -H "Authorization: Bearer <TOKEN>"
```

Listing 9: Get one student

```
curl http://localhost:8000/api/students/B00999999 \
  -H "Authorization: Bearer <TOKEN>"
```

Listing 10: Update student

```
curl -X PUT http://localhost:8000/api/students/B00999999 \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"status":"Placed"}'
```

Listing 11: Delete student (admin only)

```
curl -X DELETE http://localhost:8000/api/students/B00999999 \
  -H "Authorization: Bearer <TOKEN>"
```

11.4 Step 4: Report API Examples

Listing 12: Report e: unpaid invoices up to date

```
curl "http://localhost:8000/api/reports/unpaid-invoices?due_before
   =2026-12-31" \
  -H "Authorization: Bearer <TOKEN>"
```

Listing 13: Report g: hall student rooms

```
curl http://localhost:8000/api/reports/hall-student-rooms/1 \
  -H "Authorization: Bearer <TOKEN>"
```

Listing 14: Report n: senior staff

```
curl http://localhost:8000/api/reports/senior-staff \
  -H "Authorization: Bearer <TOKEN>"
```

12 Detailed Internal Process Flows

12.1 Flow A: Login Process

1. User submits credentials from `LoginPage`.
2. Frontend calls `apiPost("/auth/login", ...)` with `skipAuth=true`.
3. FastAPI route `/auth/login` calls `authenticate_user()`.
4. Password is verified with passlib hash.
5. On success, JWT is generated by `create_access_token()`.
6. Frontend persists token + profile and sets API auth token in memory.

12.2 Flow B: Generic Create Operation

1. Request hits `POST /api/{entity}`.
2. Entity key is validated against `TABLE_CONFIG`.
3. SQLAlchemy reflected table is fetched with `get_table()`.
4. Payload is sanitized and type-normalized by column type.
5. SQL `INSERT` is executed.
6. `_safe_commit()` commits transaction or rolls back on error.
7. Created row is re-fetched and returned.

12.3 Flow B1: Generic List Route for Each Entity (`GET /api/{entity}`)

Exact backend implementation:

Listing 15: Generic list route used for every entity

```
@app.get("/api/{entity}")
def list_records(
```

```
    entity: str,
    _: AuthUser = Depends(READ_ROLES),
    session: Session = Depends(get_session),
) -> Any:
    config = _entity_config(entity)
    table = get_table(config["table"])

    query = select(table)
    pk_name = config["pk"]
    if pk_name in table.c:
        query = query.order_by(table.c[pk_name])

    rows = session.execute(query).mappings().all()
    return _encode([dict(row) for row in rows])
```

Entity-specific behavior is derived from `TABLE_CONFIG`:

Entity URL Key	DB Table	PK Used for Sort	ID Type	Runtime Result in List Route
staff	staff	staff_id	int	`SELECT * FROM staff ORDER BY staff_id`
courses	courses	course_number	str	`SELECT * FROM courses ORDER BY course_number`
students	students	banner_id	str	`SELECT * FROM students ORDER BY banner_id`
halls	halls	hall_id	int	`SELECT * FROM halls ORDER BY hall_id`
apartments	apartments	apartment_id	int	`SELECT * FROM apartments ORDER BY apartment_id`
rooms	rooms	place_number	int	`SELECT * FROM rooms ORDER BY place_number`
leases	leases	lease_id	int	`SELECT * FROM leases ORDER BY lease_id`
invoices	invoices	invoice_id	int	`SELECT * FROM invoices ORDER BY invoice_id`
next-of-kin	next_of_kin	kin_id	int	`SELECT * FROM next_of_kin ORDER BY kin_id`
inspections	inspections	inspection_id	int	`SELECT * FROM inspections ORDER BY inspection_id`

Step-by-step runtime behavior for each request:

1. `entity` path value is validated by `_entity_config(entity)`.
2. Table name and PK are resolved from `TABLE_CONFIG`.
3. Reflected table object is loaded via `get_table(config["table"])`.
4. `select(table)` query object is created.

5. PK-based deterministic ordering is appended: `ORDER BY <pk>`.
6. SQL is executed with `session.execute(query)`.
7. Rows are materialized as mapping dictionaries.
8. Payload is JSON-encoded via `_encode(...)` and returned to frontend.

12.4 Flow C: Report Query Execution

1. Frontend chooses report metadata from `reports.ts`.
2. Endpoint is built (path and query params).
3. FastAPI report route runs SQL text against a view.
4. `session.execute(text(sql), params)` returns row mappings.
5. Rows are converted to dictionaries and encoded as JSON.

13 How Type Normalization and Validation Work

Before writes, backend coerces values according to SQL type:

- Date columns: strict `YYYY-MM-DD`
- DateTime columns: ISO datetime parsing
- Boolean columns: true/false, 1/0, yes/no parsing
- Integer and numeric fields: explicit casting

If parsing fails, API returns 400 with field-specific message.

14 Error Handling Strategy

14.1 Constraint and Integrity Mapping

`_map_integrity_error` maps common MySQL errors:

- 1062: duplicate key → 409 conflict
- 1452: invalid foreign key reference → 400 bad request
- 3819 / 1644 / 1406: validation or trigger issues → 400 bad request

14.2 Transaction Safety

`_safe_commit()` guarantees:

- commit on success
- rollback on any SQLAlchemy error
- mapped HTTP errors returned to client

15 ACID Properties: Theory and Implementation in This Project

15.1 Important Clarification

ACID is guaranteed by the database engine (MySQL/InnoDB), not by SQLAlchemy alone. SQLAlchemy provides transaction management APIs that correctly use DB transactions.

15.2 A - Atomicity

- Each write request is transactional.
- If any step fails, rollback happens, so partial writes are avoided.

15.3 C - Consistency

- Enforced by schema constraints, FKs, checks, and triggers.
- Invalid states (e.g., room with both hall and apartment) are blocked.

15.4 I - Isolation

- MySQL transaction isolation is applied per connection/session.
- Concurrent transactions are isolated according to DB isolation level.

15.5 D - Durability

- Once commit succeeds, data persists in MySQL storage engine.
- Server restarts do not lose committed data.

15.6 Practical Note on SQLAlchemy

SQLAlchemy provides:

- session boundaries
- commit and rollback orchestration
- connection pooling and retries

But data durability and strict ACID guarantees are still fundamentally DB responsibilities.

16 Code-Level Examples from Real Project

16.1 FastAPI Route Example

Listing 16: Report route structure

```
@app.get("/api/reports/hall-managers")
def report_hall_managers(
    _: AuthUser = Depends(READ_ROLES),
    session: Session = Depends(get_session),
) -> Any:
    sql = """
        SELECT hall_id, hall_name, manager_name, manager_phone
        FROM v_hall_managers
```

```
        ORDER BY hall_id
    """
    return _encode(_run_report(session, sql))
```

16.2 Generic CRUD Example

Listing 17: List route used for all entities

```
@app.get("/api/{entity}")
def list_records(entity: str, _: AuthUser = Depends(READ_ROLES),
   session: Session = Depends(get_session)) -> Any:
    config = _entity_config(entity)
    table = get_table(config["table"])
    query = select(table)
    rows = session.execute(query).mappings().all()
    return _encode([dict(row) for row in rows])
```

16.3 Frontend API Wrapper Example

Listing 18: Single API helper used by all pages

```
export async function apiRequest<T>(path: string, init:
   ApiRequestOptions = {}): Promise<T> {
  const { skipAuth = false, ...requestInit } = init;
  const headers = new Headers(requestInit.headers ?? {});

  if (!skipAuth && authToken && !headers.has("Authorization")) {
    headers.set("Authorization", `Bearer ${authToken}`);
  }

  const response = await fetch(buildUrl(path), { ...requestInit,
     headers });
  const payload = await parseResponse(response);
  if (!response.ok) {
    throw new ApiError(response.status, asErrorMessage(payload, `
       Request failed with status ${response.status}`));
  }
  return payload as T;
}
```

17 Report Layer and DBMS Demonstration Value

This project is strong for DBMS presentation because report logic is encoded in SQL views and then consumed by API endpoints. This allows demonstration in both places:

- Workbench: direct `SELECT * FROM v_...`
- Application: authenticated `/api/reports/...`

18 Deep Execution Pipeline: From `app.get(...)` to `session.execute(...)`

18.1 ASGI-to-Route Lifecycle (Exact Runtime Sequence)

For every request handled by a FastAPI route (for example `@app.get("/api/reports/hall-managers")`), the runtime path is:

1. Uvicorn receives HTTP request and forwards it to the FastAPI ASGI app.
2. Starlette/FastAPI router matches path and method to a concrete handler.
3. FastAPI builds dependency graph for that handler.
4. Security dependency extracts bearer token (if endpoint is protected).
5. JWT is decoded and role is validated by `require_roles(...)`.
6. DB dependency `get_session()` yields a SQLAlchemy `Session`.
7. Route function executes Python logic and builds SQL expression/text.
8. `session.execute(...)` sends SQL through engine to PyMySQL driver.
9. MySQL executes query and returns rows/metadata.
10. Result is materialized with `.mappings().all()` or `.first()`.
11. Backend encodes payload through `jsonable_encoder` and returns HTTP response.
12. Session dependency finalizer closes DB session.

18.2 How `app.get(...)` is Called Internally

When a route is declared with `@app.get("/api/...")`, FastAPI registers metadata for method-/path/handler. At runtime, the decorator itself is not called again; instead, routing metadata is used to dispatch incoming requests to the associated function.

18.3 CRUD Query Object Construction (SQLAlchemy Core)

In generic CRUD routes, backend builds SQL objects before execution. Example pattern:

Listing 19: SQLAlchemy query object pipeline in CRUD

```
config = _entity_config(entity)
table = get_table(config["table"])

query = select(table)
pk_name = config["pk"]
if pk_name in table.c:
    query = query.order_by(table.c[pk_name])

rows = session.execute(query).mappings().all()
```

Here, `query` is a SQLAlchemy expression object. It is compiled by the selected SQL dialect and then passed to the DBAPI driver at execution time.

18.4 Report Query Construction (Textual SQL + Bind Params)

In report routes, backend keeps SQL explicit and uses bind parameters:

Listing 20: Report execution helper path

```
def _run_report(session: Session, sql: str, params: dict[str, Any] |
    None = None):
    rows = session.execute(text(sql), params or {}).mappings().all()
    return [dict(row) for row in rows]
```

This path preserves query readability for DBMS demos while still using SQLAlchemy session management and parameter binding.

18.5 Write Operations and Transactions

For `POST/PUT/DELETE`, the execution pattern is:

1. Build `insert/update/delete` object.
2. Execute with `session.execute(...)`.
3. Call `_safe_commit()`.
4. On integrity/SQL errors: rollback + mapped HTTP exception.

19 All Views Explained in Detail (Complete Report Layer)

View	API Route	Columns Returned	Definition Logic / Source Tables
v_hall_managers	/api/reports/hall-managers	hall_id, hall_name, manager_name, manager_phone	`halls` JOIN `staff` on manager_staff_id; no filter
v_student_leases	/api/reports/student-leases	lease_id, banner_id, student_name, duration_semesters, includes_summer_semester, date_enter, date_leave, place_number, room_number, monthly_rent, residence_type, residence_name, residence_address	`leases` + `students` + `rooms` + optional `halls/apartments` joins
v_summer_leases	/api/reports/summer-leases	Same as `v_student_leases`	Derived from `v_student_leases`; filter `includes_summer_semester = TRUE`
v_student_rent_paid	/api/reports/student-rent-paid/{banner_id}	banner_id, student_name, total_paid	LEFT JOIN chain from `students -> leases -> invoices`; conditional SUM on paid invoices
v_unpaid_invoices	/api/reports/unpaid-invoices	invoice_id, lease_id, banner_id, student_name, semester, amount_due, due_date, place_number, room_number, residence_type, residence_address	`invoices` JOIN `leases/students/rooms` + optional residence joins; filter `date_paid IS NULL`
v_unsatisfactory inspections	/api/reports/ unsatisfactory-inspections	inspection_id, inspection_date, staff_name, apartment_id, comments	`inspections` JOIN `staff`; filter `is_satisfactory = FALSE`
v_hall student rooms	/api/reports/ hall-student-rooms/{hall_id}	hall_id, banner_id, student_name, hall_name, room_number, place_number	`leases` JOIN `students/rooms/halls`; hall-scoped projection

View	API Route	Columns Returned	Definition Logic / Source Tables
v_waiting_list	/api/reports/waiting-list	banner_id, first_name, last_name, street, city, postcode, mobile_phone, email, dob, gender, category, nationality, special_needs, comments, status, major, minor, course_number, advisor_staff_id	Direct projection from `students`; filter `status='Waiting'`
v_student category counts	/api/reports/ student-category-counts	category, student_count	Group-by aggregate on `students.category`
v_students without kin	/api/reports/ students-without-kin	banner_id, student_name	`students` LEFT JOIN `next_of_kin`; filter rows with null kin_id
v_student_advisers	/api/reports/student-adviser/{banner_id}	banner_id, student_name, adviser_name, adviser_phone	`students` LEFT JOIN adviser alias of `staff`
v_hall_rent_stats	/api/reports/rent-stats	min_rent, max_rent, avg_rent	Aggregate over `rooms` where `hall_id IS NOT NULL`
v_hall_place_counts	/api/reports/hall-place-counts	hall_id, hall_name, total_places	`halls` LEFT JOIN `rooms`; grouped by hall
v_senior_staff	/api/reports/senior-staff	staff_id, staff_name, age, location	`staff` with `TIMESTAMPDIFF(YEAR, dob, CURDATE()) > 60`

20 Complete Per-Table Technical Specification (All 10 Tables)

20.1 `staff`

- **Columns:** staff_id, first_name, last_name, email, street, city, postcode, dob, gender, position, department_name, internal_phone, room_number, location, created_at.
- **Primary key:** staff_id (AUTO_INCREMENT).
- **Unique constraints:** email.
- **Foreign keys (outgoing):** none.
- **Indexes:** PRIMARY(staff_id), UNIQUE(email).
- **Referenced by:** students(advisor_staff_id), halls(manager_staff_id), inspections(staff_id).

20.2 `courses`

- **Columns:** course_number, course_title, instructor_name, instructor_phone, instructor_email, instructor_room, department_name, created_at.
- **Primary key:** course_number.
- **Foreign keys (outgoing):** none.
- **Indexes:** PRIMARY(course_number).
- **Referenced by:** students(course_number).

20.3 `students`

- **Columns:** banner_id, first_name, last_name, street, city, postcode, mobile_phone, email, dob, gender, category, nationality, special_needs, comments, status, major, minor, course_number,

advisor_staff_id, created_at.

- **Primary key:** banner_id.
- **Foreign keys (outgoing):**
  - course_number -> courses(course_number), ON UPDATE CASCADE, ON DELETE SET NULL.
  - advisor_staff_id -> staff(staff_id), ON UPDATE CASCADE, ON DELETE SET NULL.
- **Indexes:** PRIMARY(banner_id), idx_students_status(status), idx_students_category(category), idx_students_advisor_staff_id(advisor_staff_id).

20.4 `halls`

- **Columns:** hall_id, hall_name, street, city, postcode, telephone, manager_staff_id, created_at.
- **Primary key:** hall_id (AUTO_INCREMENT).
- **Foreign keys (outgoing):** manager_staff_id -> staff(staff_id), ON UPDATE CASCADE, ON DELETE RESTRICT.
- **Indexes:** PRIMARY(hall_id), idx_halls_manager_id(manager_staff_id).

20.5 `apartments`

- **Columns:** apartment_id, street, city, postcode, number_of_bedrooms, created_at.
- **Primary key:** apartment_id (AUTO_INCREMENT).
- **Checks:** number_of_bedrooms IN (3,4,5).
- **Foreign keys (outgoing):** none.
- **Indexes:** PRIMARY(apartment_id).

20.6 `rooms`

- **Columns:** place_number, room_number, monthly_rent, hall_id, apartment_id, created_at.
- **Primary key:** place_number.
- **Foreign keys (outgoing):**
  - hall_id -> halls(hall_id), ON UPDATE CASCADE, ON DELETE CASCADE.
  - apartment_id -> apartments(apartment_id), ON UPDATE CASCADE, ON DELETE CASCADE.
- **Checks:** monthly_rent >= 0.
- **Triggers:** `trg_rooms_before_insert` and `trg_rooms_before_update` enforce XOR parent rule (exactly one of hall_id/apartment_id).
- **Indexes:** PRIMARY(place_number), idx_rooms_hall_id(hall_id), idx_rooms_apartment_id(apartment

20.7 `leases`

- **Columns:** lease_id, banner_id, place_number, duration_semesters, includes_summer_semester, date_enter, date_leave, created_at.
- **Primary key:** lease_id (AUTO_INCREMENT).
- **Foreign keys (outgoing):**
  - banner_id -> students(banner_id), ON UPDATE CASCADE, ON DELETE CASCADE.
  - place_number -> rooms(place_number), ON UPDATE CASCADE, ON DELETE RESTRICT.
- **Indexes:** PRIMARY(lease_id), idx_leases_banner_id(banner_id), idx_leases_place_number(place_num idx_leases_summer(includes_summer_semester).

20.8 `invoices`

- **Columns:** invoice_id, lease_id, semester, amount_due, due_date, date_paid, payment_method, first_reminder_date, second_reminder_date, created_at.
- **Primary key:** invoice_id (AUTO_INCREMENT).
- **Foreign keys (outgoing):** lease_id -> leases(lease_id), ON UPDATE CASCADE, ON DELETE CASCADE.
- **Checks:** amount_due >= 0.
- **Indexes:** PRIMARY(invoice_id), idx_invoices_lease_id(lease_id), idx_invoices_due_date(due_date), idx_invoices_date_paid(date_paid).

20.9 `next_of_kin`

- **Columns:** kin_id, banner_id, name, relationship, street, city, postcode, phone, created_at.
- **Primary key:** kin_id (AUTO_INCREMENT).
- **Unique constraints:** banner_id (one-to-one-ish from students to kin rows).
- **Foreign keys (outgoing):** banner_id -> students(banner_id), ON UPDATE CASCADE, ON DELETE CASCADE.
- **Indexes:** PRIMARY(kin_id), UNIQUE(banner_id).

20.10 `inspections`

- **Columns:** inspection_id, apartment_id, staff_id, inspection_date, is_satisfactory, comments, created_at.
- **Primary key:** inspection_id (AUTO_INCREMENT).
- **Foreign keys (outgoing):**
  - apartment_id -> apartments(apartment_id), ON UPDATE CASCADE, ON DELETE CASCADE.
  - staff_id -> staff(staff_id), ON UPDATE CASCADE, ON DELETE RESTRICT.

- **Indexes:** PRIMARY(inspection_id), idx_inspections_apartment_id(apartment_id), idx_inspections_sta idx_inspections_sat(is_satisfactory).

21 Foreign Key Matrix (All Tables)

Child Column	Parent Reference	ON UPDATE	ON DELETE	Purpose
students.course_number	courses(course_number)	CASCADE	SET NULL	Student-course mapping
students.advisor_staff_id	staff(staff_id)	CASCADE	SET NULL	Adviser relationship
halls.manager_staff_id	staff(staff_id)	CASCADE	RESTRICT	Hall manager assignment
rooms.hall_id	halls(hall_id)	CASCADE	CASCADE	Room under hall
rooms.apartment_id	apartments(apartment_id)	CASCADE	CASCADE	Room under apartment
leases.banner_id	students(banner_id)	CASCADE	CASCADE	Lease owner
leases.place_number	rooms(place_number)	CASCADE	RESTRICT	Occupancy place reference
invoices.lease_id	leases(lease_id)	CASCADE	CASCADE	Billing tied to lease
next_of_kin.banner_id	students(banner_id)	CASCADE	CASCADE	Emergency contact link
inspections.apartment_id	apartments(apartment_id)	CASCADE	CASCADE	Apartment inspection link
inspections.staff_id	staff(staff_id)	CASCADE	RESTRICT	Inspector accountability

22 Index Catalog and Query Optimization Impact

Index Name	Table	Columns	Typical API/Report Usage
PRIMARY	staff	staff_id	Staff CRUD identity lookups
UNIQUE(email)	staff	email	Avoid duplicate staff identities
PRIMARY	courses	course_number	Course CRUD identity lookups
PRIMARY	students	banner_id	Student CRUD identity lookups
idx_students_status	students	status	`v_waiting_list` filter
idx_students_category	students	category	`v_student_category_counts` grouping efficiency

Index Name	Table	Columns	Typical API/Report Usage
idx_students_advisor_staff_id	students	advisor_staff_id	Adviser joins in `v_student_advisers`
PRIMARY	halls	hall_id	Hall CRUD identity lookups
idx_halls_manager_id	halls	manager_staff_id	Hall-manager joins
PRIMARY	apartments	apartment_id	Apartment CRUD identity lookups
PRIMARY	rooms	place_number	Room CRUD identity lookups
idx_rooms_hall_id	rooms	hall_id	Hall room lookups and report generation
idx_rooms_apartment_id	rooms	apartment_id	Apartment-based joins in inspections and invoices views
PRIMARY	leases	lease_id	Lease CRUD identity lookups
idx_leases_banner_id	leases	banner_id	Student lease history, rent-paid aggregates
idx_leases_place_number	leases	place_number	Place occupancy joins
idx_leases_summer	leases	includes_summer_semester	Summer lease filtering
PRIMARY	invoices	invoice_id	Invoice CRUD identity lookups
idx_invoices_lease_id	invoices	lease_id	Lease-invoice join path
idx_invoices_due_date	invoices	due_date	Due-date filtering in unpaid invoices report
idx_invoices_date_paid	invoices	date_paid	Paid/unpaid partition scans
PRIMARY	next_of_kin	kin_id	Next-of-kin CRUD identity lookups
UNIQUE(banner_id)	next_of_kin	banner_id	Enforces max one kin row per student
PRIMARY	inspections	inspection_id	Inspection CRUD identity lookups
idx_inspections_apartment_id	inspections	apartment_id	Apartment inspection history joins
idx_inspections_staff_id	inspections	staff_id	Inspector-based filtering
idx_inspections_sat	inspections	is_satisfactory	Unsatisfactory inspection report filter

23 Deployment and Runtime Notes

23.1 Local Execution

1. Apply schema to MySQL and optional seed.

2. Configure backend `backend/.env`.
3. Run backend with Uvicorn and frontend with Vite.

23.2 Cloud Split Deployment

- Frontend: Vercel or Netlify
- Backend: Render or Railway
- Database: MySQL provider (Aiven / Railway / other managed MySQL)

23.3 TLS and Managed DB Logs

Startup messages such as "ready for connections" and "TLS enabled" indicate healthy DB startup. Self-signed CA warning is common for managed instances and handled by TLS mode/cert configuration.

24 Testing and Quality Assurance

24.1 Route-Level Tests

```
source .venv/bin/activate
PYTHONPATH=backend python -m pytest backend/tests -q
```

24.2 Runtime Smoke Test

```
source .venv/bin/activate
python backend/tests/runtime_smoke.py
```

24.3 What Is Validated

- endpoint availability
- auth behavior and token flow
- CRUD/report response correctness

25 Known Constraints and Future Improvements

- Current CORS policy is open; production should restrict origins.
- Demo users are in-memory seeded; production should store users in DB.
- Add stored procedures/functions if rubric explicitly requires them.
- Add advanced auditing, pagination, and rate limiting.

26 Final Conclusion

This project is a complete DBMS-focused software system that demonstrates relational modeling, integrity controls, transactional correctness, secure service APIs, and practical frontend integration.

From first user action to committed row, the flow is fully traceable:

1. typed frontend interaction
2. authenticated API call
3. validated backend logic
4. transactional SQL execution
5. consistent and durable relational state

It therefore satisfies both engineering and academic DBMS expectations: normalized schema design, ACID-aware operations, SQL reporting via views, and end-to-end application delivery.

A Full Report Endpoint Quick List

```
GET /api/reports/hall-managers
GET /api/reports/student-leases
GET /api/reports/summer-leases
GET /api/reports/student-rent-paid/{banner_id}
GET /api/reports/unpaid-invoices?due_before=YYYY-MM-DD
GET /api/reports/unsatisfactory-inspections
GET /api/reports/hall-student-rooms/{hall_id}
GET /api/reports/waiting-list
GET /api/reports/student-category-counts
GET /api/reports/students-without-kin
GET /api/reports/student-adviser/{banner_id}
GET /api/reports/rent-stats
GET /api/reports/hall-place-counts
GET /api/reports/senior-staff
```

B Generic CRUD Pattern Quick List

```
GET    /api/{entity}
GET    /api/{entity}/{record_id}
POST   /api/{entity}
PUT    /api/{entity}/{record_id}
DELETE /api/{entity}/{record_id}
```